INSTITUTO FEDERAL DO ESPÍRITO SANTO
MESTRADO PROFISSIONAL EM COMPUTAÇÃO APLICADA

**FLÁVIO MIOZZI BATISTA**

# LABORATÓRIO 01: LINGUAGENS REGULARES

Serra
2026

**Sumário**


**1 INTRODUÇÃO . . . 3**

**2 PARTE 1 — CONVERSÃO DE AUTÔMATOS FINITOS . . . 5**

2.1 Visão Geral do Algoritmo . . . 5

2.2 Exemplo 1 — `nfae_simple` ($L = 0^*1$) . . . 5

2.3 Exemplo 2 — `nfae_02` ($L = \{a, b\}$) . . . 6

2.4 Exemplo 3 — `nfae_04` ($L = \{ab\}$) . . . 6

2.5 Exemplo 4 — `nfae_06` ($L = (a|b)^*$) . . . 7

2.6 Exemplo 5 — `nfae_complex` . . . 7

**3 PARTE 2 — EXPRESSÕES REGULARES . . . 9**

3.1 Visão Geral — Construção de Thompson . . . 9

3.2 Expressão 1 — `ab*` . . . 9

3.3 Expressão 2 — `a*b+` . . . 9

3.4 Expressão 3 — `(a|b)*abb` . . . 10

3.5 Expressão 4 — `a?` . . . 10

3.6 Expressão 5 — `(a|b)*c(a+b|ba?)*` . . . 10

**4 PARTE 3 — ATIVIDADE COMPLEMENTAR: REGEX CROSSWORD . . . 13**

4.1 Introdução . . . 13

4.2 Desafios da Seção *Beginner* . . . 13

**4.2.1 Desafio 1 — *Beatles* . . . 13**

**4.2.2 Desafio 2 — *Naughty* . . . 14**

**4.2.3 Desafio 3 — *Ghost* . . . 15**

**4.2.4 Desafio 4 — *Symbolism* . . . 16**

**4.2.5 Desafio 5 — *Airstrip One* . . . 17**

4.3 Quebra-cabeças Criados pelo Aluno . . . 18

**4.3.1 Quebra-cabeça 1 — FLAVIO_HOUSE_IFES (Grade 5×6) . . . 18**

**4.3.2 Quebra-cabeça 2 — FLAVIO_MB_IFES (Grade Hexagonal) . . . 19**

## 1 Introdução

Este relatório documenta as três partes do Laboratório 01 da disciplina Teoria da Computação do PPCOMPI/IFES.

A **Parte 1** trata da conversão de Autômatos Finitos Não-Determinísticos com transições $\varepsilon$ (NFA$\varepsilon$) para Autômatos Finitos Determinísticos Mínimos (DFA), por meio de três etapas: remoção de transições $\varepsilon$, construção de subconjuntos e minimização pelo algoritmo de Hopcroft. A implementação encontra-se em `Main.hs`.

A **Parte 2** aborda a construção de Thompson, que converte uma expressão regular em um NFA$\varepsilon$, o qual é então convertido para DFA mínimo pelo mesmo pipeline da Parte 1. A implementação encontra-se em `Regex.hs`.

A **Parte 3** apresenta soluções para a plataforma *Regex Crossword*, que exercita a leitura e escrita de expressões regulares.

# 2 Parte 1 — Conversão de Autômatos Finitos

## 2.1 Visão Geral do Algoritmo

A conversão NFA$\varepsilon \to$ DFA mínimo implementada em `Main.hs` realiza três etapas sequenciais:

1. **Remoção de transições $\varepsilon$:** para cada estado $q$, computa-se o $\varepsilon$-fecho $\overline{\varepsilon}(q)$ — o conjunto de estados alcançáveis a partir de $q$ por zero ou mais transições $\varepsilon$. As transições do novo NFA são redefinidas incorporando esses fechos.

2. **Construção de subconjuntos (*subset construction*):** os estados do DFA são subconjuntos de estados do NFA. O DFA inicial corresponde ao $\varepsilon$-fecho do estado inicial do NFA, e as transições são computadas iterativamente até que nenhum novo subconjunto seja gerado.

3. **Minimização por Hopcroft:** o DFA resultante é minimizado usando o algoritmo de Hopcroft, que particiona os estados em classes de equivalência com base na distinguibilidade por strings de entrada.

## 2.2 Exemplo 1 — `nfae_simple` ($L = 0^*1$)

O autômato `nfae_simple` possui 3 estados sobre o alfabeto $\Sigma = \{0, 1\}$ e reconhece $L = 0^*1$ (zero ou mais 0s seguidos de exatamente um 1). A transição $\varepsilon$ de $q_0$ para $q_1$ permite que o autômato "pule" diretamente para o estado que aguarda o símbolo 1.

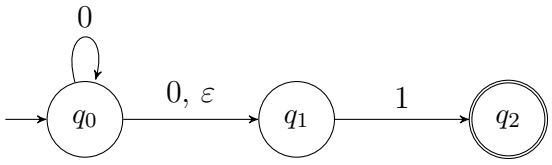



Figura 1 – NFA$\varepsilon$ `nfae_simple`: estado inicial $q_0$, estado final $q_2$.

Após a remoção das transições $\varepsilon$, o NFA equivalente consolida o comportamento de $q_0$ absorvendo as transições de $q_1$:

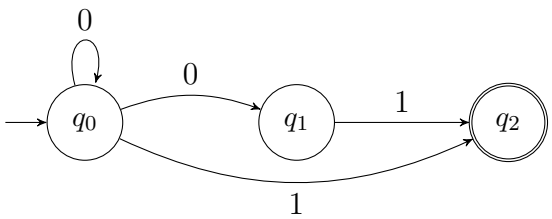



Figura 2 – NFA intermediário (sem transições $\varepsilon$).

O DFA mínimo resultante possui apenas 2 estados: o estado $\{q_0\}$ (inicial, não-final) e $\{q_2\}$ (final). O estado morto implícito, para entradas após $\{q_2\}$, é eliminado pela minimização.

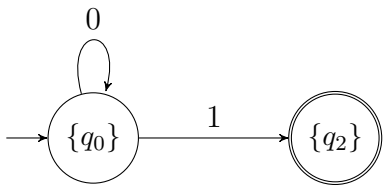



Figura 3 – DFA mínimo: $L = 0^*1$.

## 2.3 Exemplo 2 — `nfae_02` ($L = \{a, b\}$)

O autômato `nfae_02` possui 5 estados, usa transições $\varepsilon$ para modelar a escolha não-determinística entre $a$ e $b$, e reconhece $L = \{a, b\}$ (exatamente uma letra).

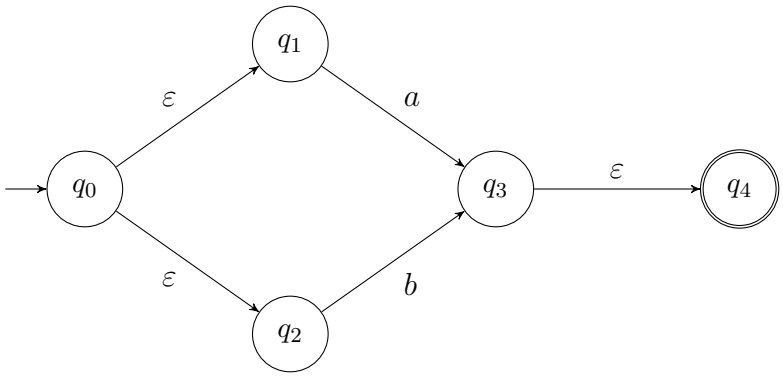



Figura 4 – NFA$\varepsilon$ `nfae_02`: $L = \{a, b\}$.

O DFA mínimo colapsa todos os estados intermediários em dois estados:

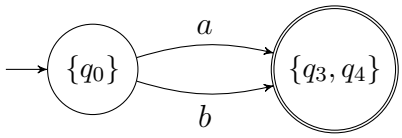



Figura 5 – DFA mínimo: $L = \{a, b\}$.

## 2.4 Exemplo 3 — `nfae_04` ($L = \{ab\}$)

O autômato `nfae_04` possui 4 estados e reconhece $L = \{ab\}$ (somente a cadeia "ab"). A transição $\varepsilon$ entre $q_1$ e $q_2$ serve como conector entre os dois estágios de leitura.

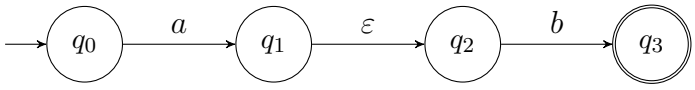



Figura 6 – NFA$\varepsilon$ `nfae_04`: $L = \{ab\}$.

Após a construção de subconjuntos, o estado $\{q_1, q_2\}$ surge porque $q_2$ pertence ao $\varepsilon$-fecho de $q_1$.

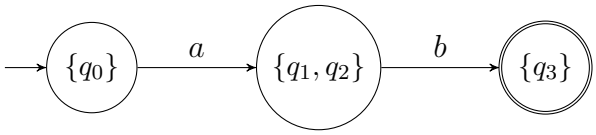



Figura 7 – DFA mínimo: $L = \{ab\}$.

## 2.5 Exemplo 4 — `nfae_06` ($L = (a|b)^*$)

O autômato `nfae_06` é a construção de Thompson para $(a|b)^*$, com 8 estados. O estado inicial $q_0$ possui transições $\varepsilon$ diretas para $q_1$ (ramo interno do Kleene) e $q_7$ (estado final, para aceitar a cadeia vazia). O ciclo $q_6 \to q_1$ implementa a repetição.

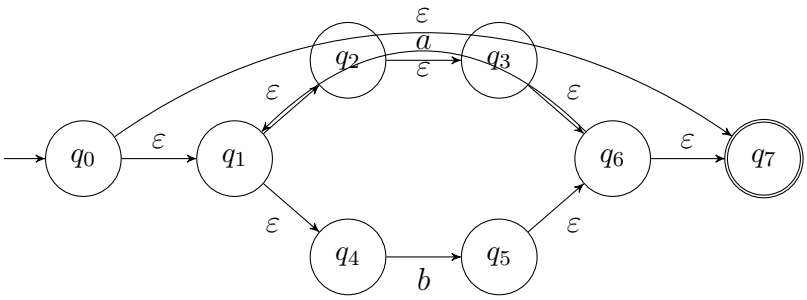



Figura 8 – NFA$\varepsilon$ `nfae_06` (construção de Thompson para $(a|b)^*$).

Como $(a|b)^*$ aceita qualquer cadeia sobre $\{a, b\}$, o DFA mínimo colapsa para um único estado, inicial e final, com auto-laços em $a$ e $b$:

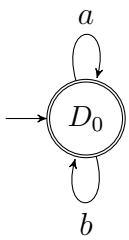



Figura 9 – DFA mínimo: $L = (a|b)^*$ (estado único, inicial e final).

## 2.6 Exemplo 5 — `nfae_complex`

O autômato `nfae_complex` possui 13 estados sobre $\Sigma = \{a, b, c\}$ e reconhece cadeias que: (i) começam com $ab$ ou $cb$; (ii) contêm ao menos um $c$ intermediário; e (iii) terminam com $ba$ ou $cc$.

A Figura 10 apresenta o diagrama do NFA$\varepsilon$, onde o par $(q_0 \to q_1 \to q_2,\ q_0 \to q_3 \to q_4) \to q_5$ codifica os prefixos alternativos, $q_6$ e $q_7$ modelam o meio da cadeia, e os sufixos $ba$ e $cc$ são capturados por $(q_8, q_9)$ e $(q_{10}, q_{11})$, respectivamente, convergindo para $q_{12}$.

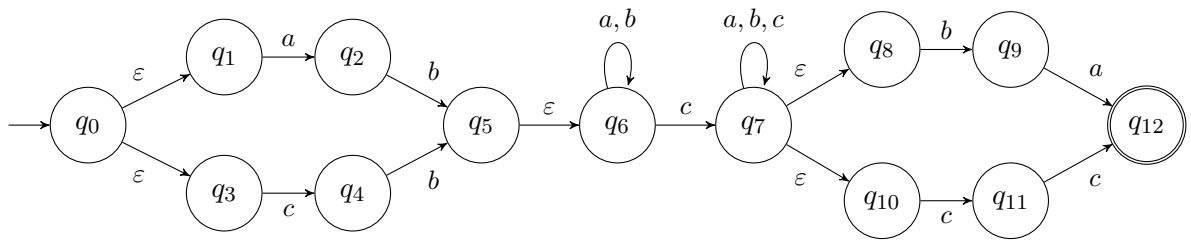



Figura 10 – NFA$\varepsilon$ `nfae_complex` (13 estados, $\Sigma = \{a, b, c\}$).

A construção de subconjuntos seguida de minimização produz um DFA com 8 estados. A Tabela 1 define a correspondência entre os estados renomeados ($D_0$–$D_7$) e os subconjuntos de estados do NFA.

Tabela 1 – Correspondência de estados — DFA de `nfae_complex`.

| **Estado DFA** | **Subconjunto NFA** | **Final?** |
|---|---|---|
| $D_0$ | $\{q_0\}$ | Não |
| $D_1$ | $\{q_2\}$ | Não |
| $D_2$ | $\{q_6\}$ | Não |
| $D_3$ | $\{q_6, q_7, q_8, q_{10}\}$ | Não |
| $D_4$ | $\{q_6, q_7, q_8, q_{10}, q_{11}\}$ | Não |
| $D_5$ | $\{q_6, q_7, q_8, q_9, q_{10}\}$ | Não |
| $D_6$ | $\{q_6, q_7, q_8, q_{10}, q_{11}, q_{12}\}$ | **Sim** |
| $D_7$ | $\{q_6, q_7, q_8, q_{10}, q_{12}\}$ | **Sim** |

Tabela 2 – Tabela de transições do DFA mínimo de `nfae_complex`. O símbolo "$\emptyset$" indica transição para o estado morto (eliminado pela minimização).

| **Estado** | **Final?** | $a$ | $b$ | $c$ |
|---|---|---|---|---|
| $\to D_0$ | | $D_1$ | $\emptyset$ | $D_1$ |
| $D_1$ | | $\emptyset$ | $D_2$ | $\emptyset$ |
| $D_2$ | | $D_2$ | $D_2$ | $D_3$ |
| $D_3$ | | $D_3$ | $D_5$ | $D_4$ |
| $D_4$ | | $D_3$ | $D_5$ | $D_6$ |
| $D_5$ | | $D_7$ | $D_5$ | $D_4$ |
| $D_6$ | $*$ | $D_3$ | $D_5$ | $D_6$ |
| $D_7$ | $*$ | $D_3$ | $D_5$ | $D_4$ |

# 3 Parte 2 — Expressões Regulares

## 3.1 Visão Geral — Construção de Thompson

O módulo `Regex.hs` implementa a conversão de uma expressão regular para DFA mínimo em quatro estágios:

1. **Análise sintática (*parsing*):** a expressão regular é lida como string e convertida para uma árvore sintática abstrata (AST) usando combinadores de *parsing* em Haskell.

2. **Construção de Thompson:** a AST é percorrida recursivamente, gerando um NFA$\varepsilon$ por indução estrutural — concatenação, alternância (|), Kleene ($*$/+) e opção (?) possuem construções padrão.

3. **Eliminação de $\varepsilon$ e construção de subconjuntos:** o NFA$\varepsilon$ gerado é passado ao mesmo pipeline de `Main.hs`.

4. **Minimização de Hopcroft:** o DFA resultante é minimizado.

Para cada expressão regular a seguir, apresenta-se a expressão, a linguagem reconhecida e o DFA mínimo obtido, com renomeação dos estados por clareza.

## 3.2 Expressão 1 — `ab*`

A expressão `ab*` denota $L = \{ab^n \mid n \geq 0\} = a \cdot b^*$: uma cadeia começa com exatamente um $a$, seguido de zero ou mais $b$s. O DFA mínimo possui 2 estados.

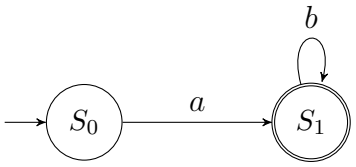



Figura 11 – DFA mínimo para `ab*`: $S_0 = \{q_0\}$, $S_1 = \{q_3, q_4, q_5\}$.

## 3.3 Expressão 2 — `a*b+`

A expressão `a*b+` denota $L = a^* \cdot b^+$: zero ou mais $a$s seguidos de ao menos um $b$. O DFA mínimo possui 2 estados.

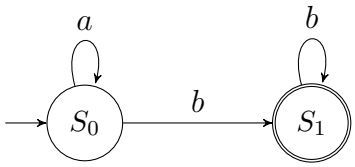



Figura 12 – DFA mínimo para `a*b+`: $S_0 = \{q_0\}$, $S_1 = \{q_7, q_8, q_9\}$.

## 3.4 Expressão 3 — `(a|b)*abb`

A expressão `(a|b)*abb` denota $L = (a|b)^* \cdot abb$: todas as cadeias sobre $\{a, b\}$ que terminam com o sufixo $abb$. Este é o exemplo clássico de 4 estados que ilustra o DFA da construção de subconjuntos.

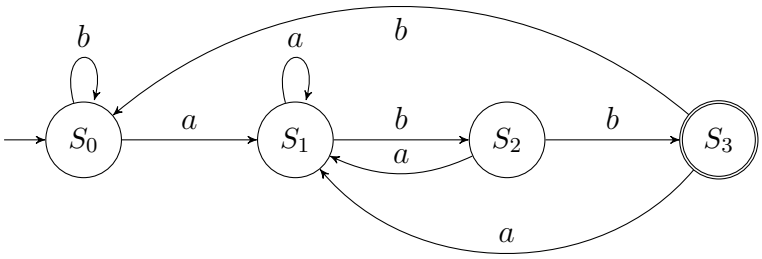



Figura 13 – DFA mínimo para `(a|b)*abb`. $S_0 = \{q_0\}$; $S_1 = \{q_1, q_2, q_3, q_4, q_5, q_6, q_8, q_9, q_{10}\}$; $S_2 = \{q_1, q_2, q_3, q_4, q_6, q_7, q_8, q_{11}, q_{12}\}$; $S_3 = \{q_1, q_2, q_3, q_4, q_6, q_7, q_8, q_{13}\}^*$.

## 3.5 Expressão 4 — `a?`

A expressão `a?` denota $L = \{\varepsilon, a\}$: a cadeia vazia ou exatamente um $a$. Como o alfabeto de entrada é $\{a\}$, o DFA mínimo possui 2 estados, ambos finais (a cadeia vazia é aceita pelo estado inicial).

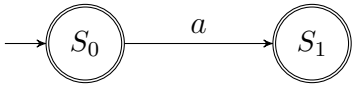



Figura 14 – DFA mínimo para `a?`: $S_0 = \{q_0\}$, $S_1 = \{q_1, q_3\}$. Ambos os estados são finais; $S_1$ não possui transições saindo pois não há símbolo $a$ disponível após consumir um $a$.

## 3.6 Expressão 5 — `(a|b)*c(a+b|ba?)*`

A expressão `(a|b)*c(a+b|ba?)*` denota cadeias sobre $\{a, b, c\}$ compostas de: (i) qualquer prefixo em $(a|b)^*$; (ii) exatamente um $c$; e (iii) um sufixo arbitrário em $(a^+b \mid ba^?)^*$. O DFA mínimo possui 4 estados (Tabela 3).

Tabela 3 – DFA mínimo para `(a|b)*c(a+b|ba?)*`. Estados finais: $R_1$, $R_2$.

| **Estado** | **Final?** | $a$ | $b$ | $c$ |
|---|---|---|---|---|
| $\rightarrow R_0$ | | $R_0$ | $R_0$ | $R_1$ |
| $R_1$ | $*$ | $R_3$ | $R_2$ | — |
| $R_2$ | $*$ | $R_1$ | $R_2$ | — |
| $R_3$ | | $R_3$ | $R_1$ | — |

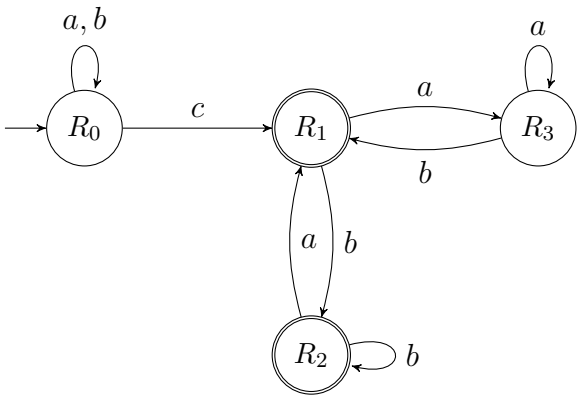



Figura 15 – DFA mínimo para `(a|b)*c(a+b|ba?)*`. $R_0$=$\{q_0\}$; $R_1$=$\{q_9, q_{10}, q_{11}, q_{12}, q_{14}, q_{22}\}^*$; $R_2$=$\{q_{11}, q_{12}, q_{13}, q_{14}, q_{22}, q_{23}, \ldots, q_{29}\}^*$; $R_3$=$\{q_{17}, q_{18}, q_{19}, q_{20}\}$.

# 4 Parte 3 — Atividade Complementar: Regex Crossword

## 4.1 Introdução

*Regex Crossword* é uma plataforma de quebra-cabeças interativos disponível em `https://regexcrossword.com`, na qual o objetivo é preencher uma grade bidimensional com caracteres de modo que cada linha e cada coluna satisfaça simultaneamente a expressão regular especificada como restrição. A atividade exercita, de maneira lúdica, a capacidade do aluno de construir e interpretar expressões regulares — habilidade central na disciplina de Teoria da Computação.

Este capítulo apresenta as soluções para os cinco desafios da seção *Beginner* e, em seguida, descreve dois quebra-cabeças originais criados e publicados pelo aluno na seção *Player Puzzles*.

## 4.2 Desafios da Seção *Beginner*

A seção *Beginner* compreende cinco desafios introdutórios, cada um com grade 2×2. A Figura 16 apresenta a visão geral dos cinco desafios concluídos.

Figura 16 – Visão geral dos cinco desafios da seção *Beginner* concluídos: Beatles, Naughty, Ghost, Symbolism e Airstrip One.

### 4.2.1 Desafio 1 — *Beatles*

O desafio *Beatles* apresenta restrições baseadas em alternância e classes de caracteres. A solução é formada pelas letras **H**, **E**, **L**, **P** — compondo a palavra *HELP*, título do álbum homônimo dos Beatles de 1965.

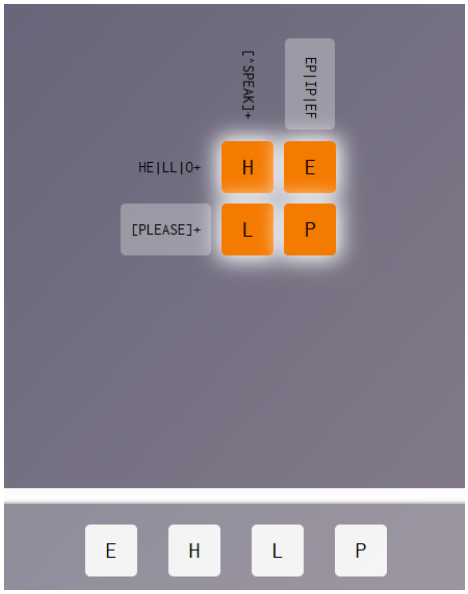



Figura 17 – Desafio *Beatles* — solução: HELP.

| | HE\|LL\|O+ | [PLEASE]+ |
|---|---|---|
| [*SPEAK]+ | **H** | **E** |
| EP\|IP\|EF | **L** | **P** |

### 4.2.2 Desafio 2 — *Naughty*

O desafio *Naughty* introduz o operador de referência reversa (`\1`), que exige que determinada subcadeia coincida com o grupo de captura anterior. A solução é **B**, **O**, **B**, **E**.

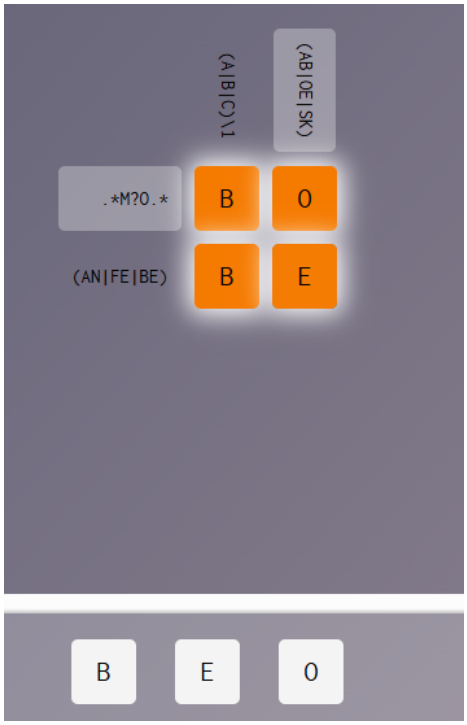



Figura 18 – Desafio *Naughty* — solução: BOBE.

| | `(A\|B\|C)\1` | `(AB\|OE\|SK)` |
|---|---|---|
| `.*M?O.*` | **B** | **O** |
| `(AN\|FE\|BE)` | **B** | **E** |

Verificação das restrições:

- Linha 1 (*BO*): `.*M?O.*` — `B` casa `.*`, `M?` é omitido (zero ocorrências), `O` casa literalmente ✓
- Linha 2 (*BE*): `(AN|FE|BE)` — terceira alternativa ✓
- Coluna 1 (*BB*): `(A|B|C)\1` — grupo captura `B`, referência reversa exige novo `B` ✓
- Coluna 2 (*OE*): `(AB|OE|SK)` — segunda alternativa ✓

#### 4.2.3 Desafio 3 — *Ghost*

No desafio *Ghost*, todas as quatro células recebem o caractere **O**, compondo a solução *OOOO*.

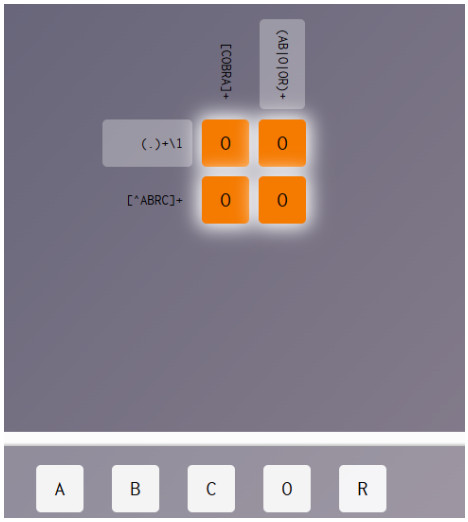



Figura 19 – Desafio *Ghost* — solução: OOOO.

| | `[COBRA]+` | `(AB\|O\|OR)+` |
|---|---|---|
| `(.)+\1` | **O** | **O** |
| `[^ABRC]+` | **O** | **O** |

Verificação:

- Linha 1 (*OO*): `(.)+\1` — `(.)` captura `O`, quantificador `+` permite iteração, referência reversa exige novo `O` ✓
- Linha 2 (*OO*): `[^ABRC]+` — `O` $\notin$ {A, B, R, C} ✓
- Coluna 1 (*OO*): `[COBRA]+` — `O` $\in$ {C, O, B, R, A} ✓
- Coluna 2 (*OO*): `(AB|O|OR)+` — alternativa `O` aplicada duas vezes ✓

#### 4.2.4 Desafio 4 — *Symbolism*

O desafio *Symbolism* utiliza exclusivamente caracteres simbólicos do alfabeto {`*`, `+`, `,`, `/`}. A solução é `*`, `*`, `/`, `/`.

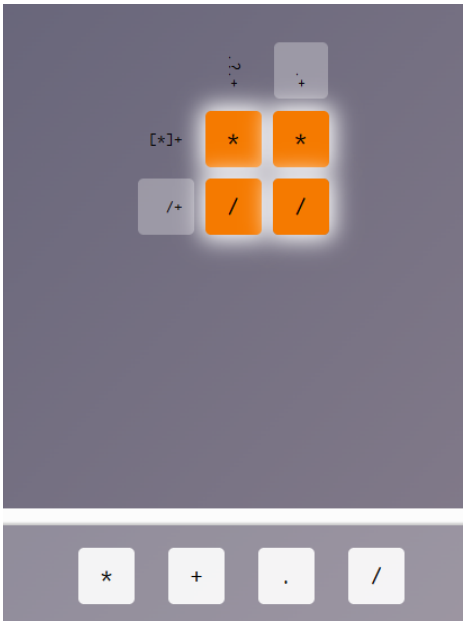



Figura 20 – Desafio *Symbolism* — solução: `**//` .

| | |
|---|---|
| * | * |
| / | / |

O desafio explora o fato de que, dentro de classes de caracteres (`[...]`), quantificadores como `*` e `+` perdem seu significado especial e são interpretados como literais. A restrição de linha 2, `/+`, exige um ou mais `/` — a sequência // satisfaz a expressão ✓.

#### 4.2.5 Desafio 5 — *Airstrip One*

O desafio *Airstrip One* utiliza dígitos do alfabeto {0, 1, 4, 6, 8, 9}. A solução é **1**, **9**, **8**, **4** — compondo o número *1984*, referência ao romance distópico de George Orwell (*Airstrip One* é o nome dado à Inglaterra na obra).

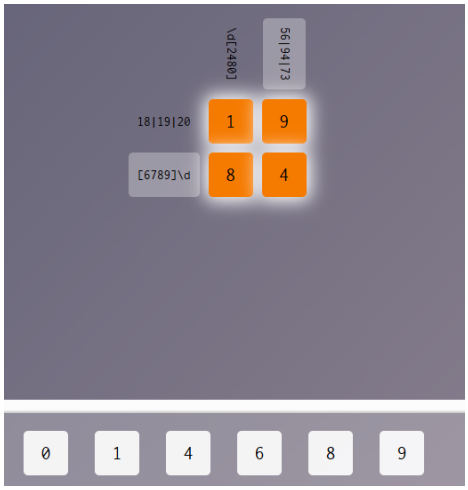



Figura 21 – Desafio *Airstrip One* — solução: 1984.

| | \d[2480] | 56\|94\|73 |
|---|---|---|
| 18\|19\|20 | **1** | **9** |
| [6789]\d | **8** | **4** |

Verificação:

- Linha 1 (*19*): `18|19|20` — segunda alternativa ✓
- Linha 2 (*84*): `[6789]\d` — `8` $\in \{6,7,8,9\}$, `4` $\in \Sigma_{\text{dígitos}}$ ✓
- Coluna 1 (*18*): `\d[2480]` — `1` $\in \Sigma_{\text{dígitos}}$, `8` $\in \{2,4,8,0\}$ ✓
- Coluna 2 (*94*): `56|94|73` — segunda alternativa ✓

## 4.3 Quebra-cabeças Criados pelo Aluno

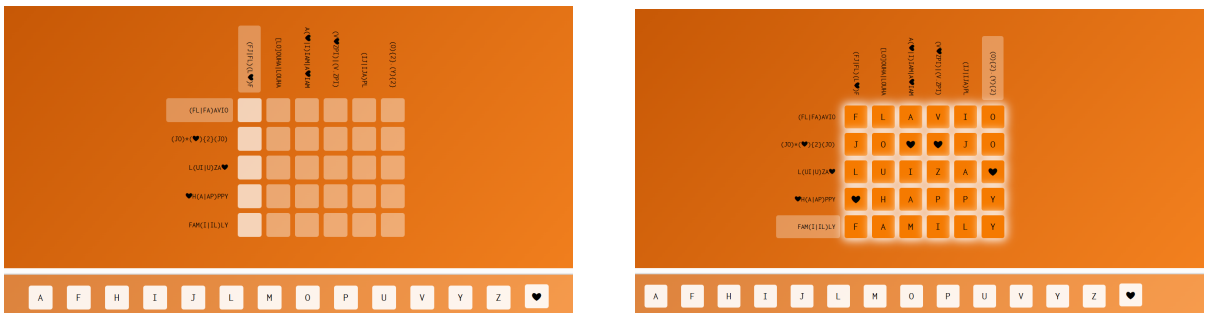
Dois quebra-cabeças originais foram criados e publicados na seção *Player Puzzles* do *Regex Crossword*. Ambos empregam referências ao nome do autor e à instituição de ensino.

### 4.3.1 Quebra-cabeça 1 — FLAVIO_HOUSE_IFES (Grade 5×6)

O quebra-cabeça `FLAVIO_HOUSE_IFES` é uma grade retangular de 5 linhas por 6 colunas, disponível em:

`https://regexcrossword.com/playerpuzzles/69ec93119f2fd`

As expressões regulares de linha foram projetadas para evocar palavras associadas ao nome do autor, à família e ao lar. O alfabeto utilizado inclui letras maiúsculas e o símbolo ♡ como caractere especial.



Quebra-cabeça sem solução. Quebra-cabeça com solução preenchida.

Figura 22 – Quebra-cabeça FLAVIO_HOUSE_IFES — grade retangular 5×6.

As restrições de linha projetadas são apresentadas na Tabela 4.

Tabela 4 – Restrições de linha do quebra-cabeça FLAVIO_HOUSE_IFES.

| Linha | Expressão Regular | Solução |
|---|---|---|
| 1 | `(FL\|FA)AVIO` | FLAVIO |
| 2 | `(JO)+♡(2)(JO)` | JO♡JO |
| 3 | `L(UI\|U)ZA♡` | LUZA♡ |
| 4 | `♡H(A\|AP)PPY` | ♡HAPPY |
| 5 | `FAM(I\|IL)Y` | FAMILY |

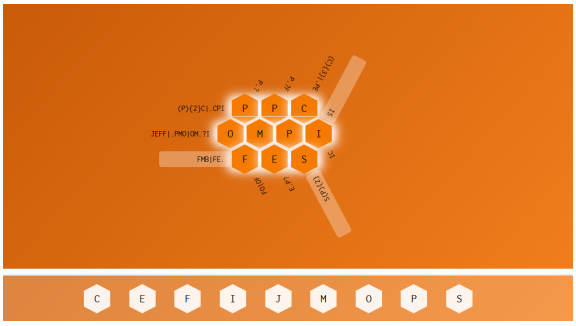
### 4.3.2 Quebra-cabeça 2 — FLAVIO_MB_IFES (Grade Hexagonal)

O quebra-cabeça `FLAVIO_MB_IFES` é uma grade hexagonal de tamanho 10 (arranjo 3-4-3), disponível em:

`https://regexcrossword.com/playerpuzzles/69ec8bdfb48b0`

A grade hexagonal impõe restrições em três direções — linhas horizontais e duas diagonais — o que aumenta significativamente a complexidade do design em relação a grades retangulares. O alfabeto utilizado é {C, E, F, I, J, M, O, P, S}.

Listagem do quebra-cabeça.




Quebra-cabeça com solução preenchida.

Figura 23 – Quebra-cabeça FLAVIO_MB_IFES — grade hexagonal, tamanho 10.

A solução da grade hexagonal, lida linha a linha em arranjo 3-4-3, compõe:

| | | |
|---|---|---|
| Linha 1: | **P P C** | (3 células) |
| Linha 2: | **O M P I** | (4 células) |
| Linha 3: | **F E S** | (3 células) |

O resultado — *PPCOMPIFES* — é uma referência ao Programa de Pós-Graduação em Computação Aplicada (PPCOMPI) do Instituto Federal do Espírito Santo (IFES), programa ao qual o autor está vinculado. As restrições de linha horizontais são apresentadas na Tabela 5.

Tabela 5 – Restrições de linha do quebra-cabeça FLAVIO_MB_IFES.

| **Linha** | **Expressão Regular** | **Solução** |
|---|---|---|
| 1 | `(P)(2)C|CPI` | PPC |
| 2 | `JEFF|PMO|OM?I` | OMPI |
| 3 | `FMB|FE.` | FES |